

1.2 Coordinate Systems



Core skills you will master in this section:

- ▶ The real number line and the essence of one-dimensional Euclidean space: the three key elements of the real number line — the origin, the positive direction, and the unit length.
- ▶ The Cartesian coordinate plane: methods for visualizing two- and three-dimensional vectors, and the right-hand-rule convention for direction.
- ▶ Using `numpy.meshgrid()` to create two- and three-dimensional grid data: how to generate grid data as a foundation for plotting and numerical computation.
- ▶ Representing and visualizing two-dimensional vectors in the Cartesian coordinate plane.
- ▶ A first look at span and the idea behind vector spaces: building an intuitive understanding of vector spaces through unit vectors that span a space.
- ▶ Computing the length of a vector: mastering how the magnitude of a vector is calculated, verified using the Pythagorean theorem.

To visualize vectors, we need the help of coordinate systems. By introducing the Cartesian plane and three-dimensional Cartesian coordinates, we can make abstract vectors concrete, giving them a definite position and direction in geometric space. This section starts with the most basic idea, the real number line, and examines its essence as a one-dimensional Euclidean space. It then introduces the Cartesian coordinate plane, explaining the origins of the Cartesian coordinate system and its role as a bridge between geometry and algebra. Through the representation of vectors, we see how coordinates can describe points in space and their positional relationships. The section also explores the use of `numpy.meshgrid()`, which generates regular coordinate grid data as a foundation for numerical computation and plotting. We then extend further to three-dimensional Cartesian coordinates and emphasize how the right-hand rule governs spatial direction. Using the RGB color model, we visually illustrate vector relationships in three-dimensional coordinates, while introducing the concept of vector decomposition. Finally, we discuss how to compute the length of a vector, introduce the Euclidean norm and how it is calculated, and provide accompanying code examples.

The Real Number Line

Before introducing Cartesian coordinates, let's first get acquainted with the real number line.

As shown in Figure 1, the real number line (also called the real line or real axis) is a straight line that extends infinitely in both directions; it gives us a basic tool for representing the position of a real number (a scalar).

The origin (0), the positive direction (the way the arrow points), and the unit length (1) are the three key elements of the number line. Numbers to the left of the origin are negative, and numbers to the right are positive.

The real number line extends infinitely to the left and right, approaching negative infinity ($-\infty$) on the left and positive infinity ($+\infty$) on the right.

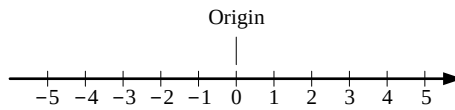


Figure 1. The Real Number Line

The real number line is also called one-dimensional Euclidean space, denoted $\mathbb{R}$, which is just the set of real numbers.

The Cartesian Coordinate Plane

The Cartesian coordinate system, also called the rectangular coordinate system, is a system that uses a set of numbers to uniquely determine the position of a point in space.

René Descartes introduced the concept of the coordinate system in the 17th century, pioneering a close union between algebra and geometry. He held that geometric figures could be described not only by their intuitive shapes, but also by numbers and symbols.

According to legend, while lying in bed pondering a mathematical problem, Descartes noticed a fly on the ceiling and realized that its position could be described using two perpendicular number lines. This flash of insight led him to invent the Cartesian coordinate system, which turns geometric problems into algebraic equations that can be solved.

This approach is precisely what allows geometric problems to be turned into algebraic ones — solving equations to find the intersection points of curves, compute distances, study the properties of shapes, and so on. Descartes's coordinate system not only brought algebraic rigor to traditional geometry, but also laid the foundation for the later development of analytic geometry, making it possible to handle complex geometric shapes with simple algebraic operations, and greatly advancing the development of mathematics as a whole.

As shown in Figure 2(a), when two number lines intersect perpendicularly at the origin, they form the Cartesian coordinate plane, also called two-dimensional Euclidean space, denoted $\mathbb{R}^2$.

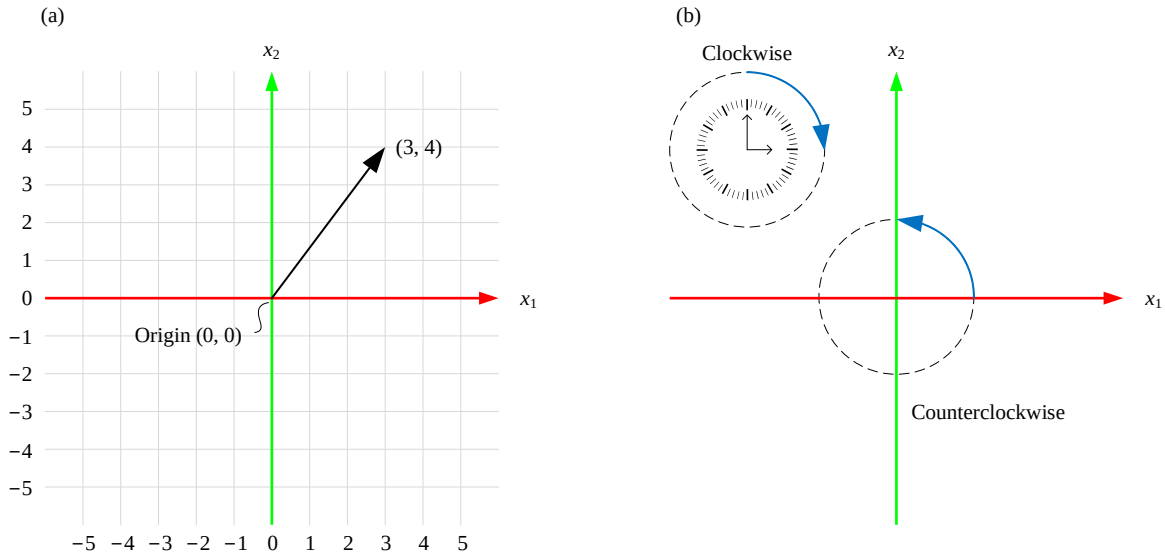


Figure 2. The Cartesian Coordinate Plane

In this coordinate system, the coordinates of every point are represented by a tuple (x, y) or (x_1, x_2) , which this book usually writes as (x_1, x_2) ; here x_1 gives the horizontal position and x_2 gives the vertical position.

A column vector $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ can be viewed as a vector in the plane whose tail is at $(0, 0)$ and whose head is at (x_1, x_2) ; clearly, the zero vector $\mathbf{0}$ sits at the origin of the coordinate system.

In general, unless stated otherwise, we assume a vector's tail is at the origin. This lets us represent a vector uniquely by its coordinate values, and lets us compute and derive results using algebraic methods.

Of course, in some settings we may translate a vector to a different position. For example, in the next section's discussion of vector addition, we will align the tail of one vector with the head of another to construct a “tip-to-tail” geometric picture, giving us an intuitive way to understand how vectors add.

You may have already noticed that in Figure 2, the x_1 axis is colored red, while the x_2 axis is colored green. This is by design: we will use scattered points on the “red-green” plane to represent the different colors produced by mixing red and green light, as a way of explaining two-dimensional vectors.

In addition, in the Cartesian plane, as Figure 2(b) shows, rotating 90 degrees from the positive x_1 direction to the positive x_2 direction is a counterclockwise rotation, and this direction is defined as positive; put another way, viewed counterclockwise (within 180 degrees), the positive x_1 direction leads the positive x_2 direction.

This means we have defined a standard right-handed orientation, so that both the direction of vectors and the sense of rotation follow the convention that “counterclockwise is positive.”

This ordering matters a great deal, because it affects many geometric and algebraic calculations — such as determinants, cross products, and the sense of an angle — ensuring that spatial relationships stay consistent and correct. For instance, swapping the order of two vectors in a cross product flips the sign of the result, and swapping two rows of a matrix flips the sign of its determinant — both are direct consequences of fixing an orientation convention like the one defined here.

Creating Two-Dimensional Grid Data

The function `numpy.meshgrid()` comes from NumPy and is used to generate coordinate grid data; such grid data is commonly used in numerical computation, plotting, and vectorized operations.

The function `numpy.meshgrid()` takes one-dimensional arrays as input and produces two or more coordinate matrices, so that in a multidimensional space, the coordinates of every point can be obtained by indexing into these matrices.

As shown in Figure 3, on a two-dimensional plane, `numpy.meshgrid()` can create a coordinate grid along an x_1 axis and an x_2 axis; what this function outputs are the x_1 and x_2 coordinate values at each grid point (both are two-dimensional arrays).

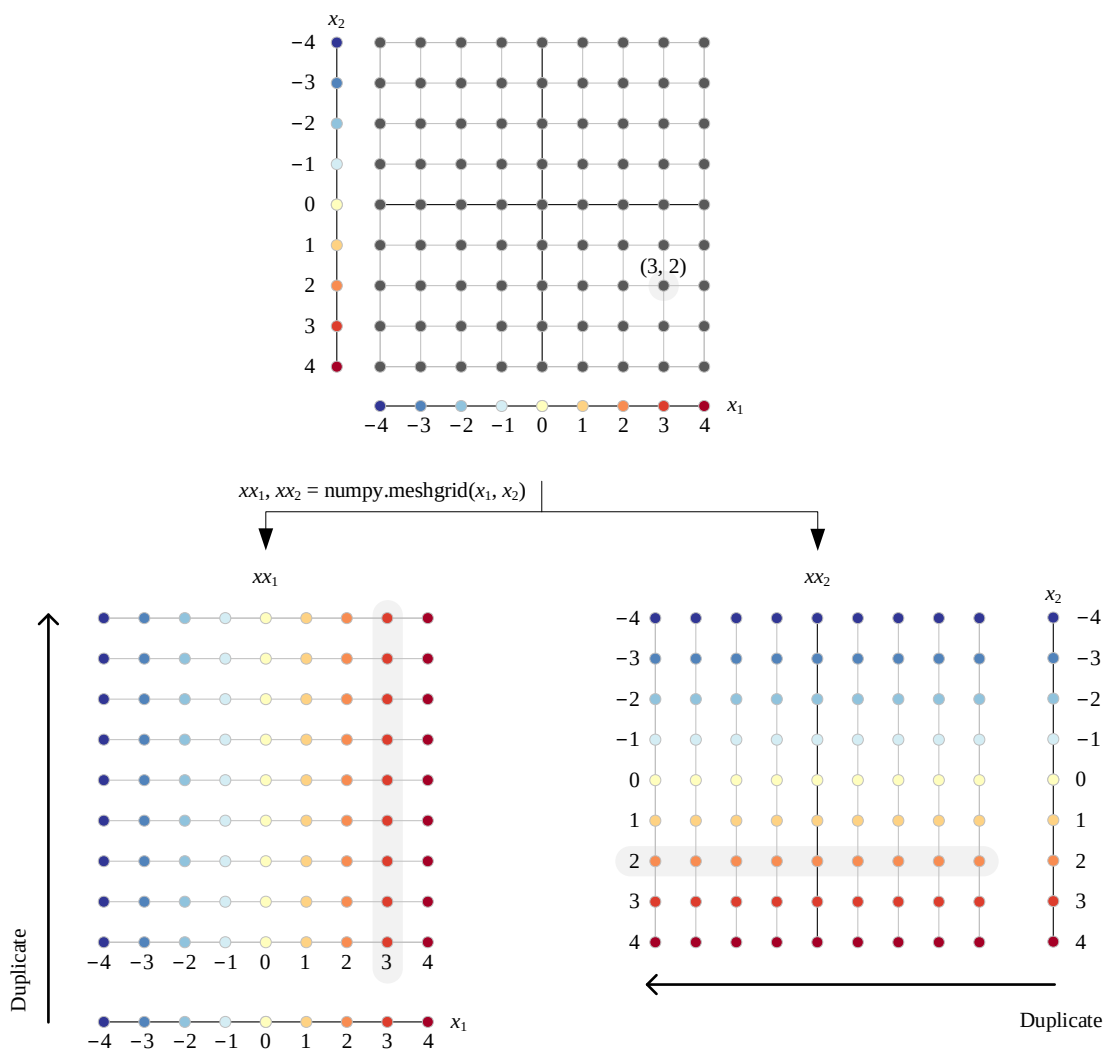


Figure 3. Creating Two-Dimensional Grid Data with `numpy.meshgrid()`

Code 1 generates the two-dimensional grid data in Figure 3.

Code 1. Creating Two-Dimensional Grid Data with `numpy.meshgrid()` |  LA_01_02_01.ipynb

This PDF file is the author's draft, published to make it convenient for readers to study on mobile devices; the final content is subject to the printed edition published by Tsinghua University Press.

Copyright belongs to Tsinghua University Press. Please do not use it commercially, and please credit the source when quoting.

Code and PDF file downloads: <https://github.com/Visualize-ML>

The companion video lectures for this book are all published on Bilibili — 生姜 DrGinger: <https://space.bilibili.com/513194466>

Comments and corrections are welcome. The book's dedicated email address: jiang.visualize.ml@gmail.com

```

a import numpy as np

# 生成 x1 和 x2 的一维数组，范围 [-4, 4]，步长为 1
b x1 = np.arange(-4, 5, 1)
  x2 = np.arange(-4, 5, 1)

# 生成网格
c xx1, xx2 = np.meshgrid(x1, x2)

```

Let's now go through Code 1 sentence by sentence.

a imports the NumPy library and gives it the alias `np`, so that later code can call NumPy's functions using `np` instead of typing out `numpy` in full each time — making the code more concise and readable.

b uses the `numpy.arange()` function to create a one-dimensional array. `numpy.arange(start, stop, step)` generates an array of evenly spaced values (spaced by `step`) from `start` up to `stop`, but `stop` itself is not included in the result.

c uses the `numpy.meshgrid()` function, which takes the two one-dimensional arrays `x1` and `x2` and produces two two-dimensional arrays, `xx1` and `xx2`, that store the coordinate values of every point in the grid. `xx1` holds the `x1`-coordinate of every point in the grid, and `xx2` holds the `x2`-coordinate.

As shown in Figure 3, geometrically, `xx1` amounts to copying the one-dimensional array `x1` across every row, while `xx2` amounts to copying the one-dimensional array `x2` down every column, together forming a regular two-dimensional grid.

Note that, vertically, in the `xx2` array produced by `numpy.meshgrid()`, negative numbers appear at the top and positive numbers at the bottom — the opposite of a typical Cartesian coordinate diagram. In a standard mathematical plot, the positive direction of the vertical axis points upward, but the `xx2` array from `numpy.meshgrid()` follows array-index order, so its values increase from top to bottom.

Placing 2D Vectors in the Cartesian Plane

When there are only a few vectors, or when we care about the exact position of a vector's tail and head, we typically use an arrow (or quiver) to represent it.

When there are many vectors, or when we mainly care about their overall distribution and trend, we tend to use a scatter plot for a simpler representation; here every vector's tail is assumed to be the origin, and its head is indicated by the position of the corresponding scatter point.

As shown in Figure 4(a), the red vector is $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$, representing pure red (light); the green vector is $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$, representing pure green (light).

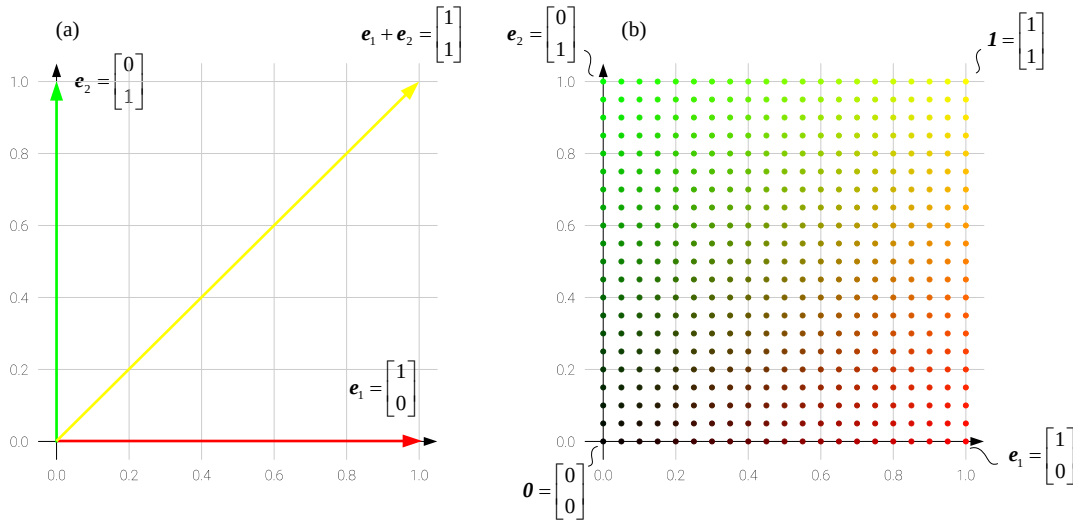


Figure 4. Unit Vectors and a Regular Scatter Plot on the “Red-Green” Plane

As shown in Figure 4(a), on the red-green plane (the x_1x_2 plane), the origin $\begin{bmatrix} 0 \\ 0 \end{bmatrix}$ (the zero vector $\mathbf{0}$) represents pure black.

As mentioned in the previous section, a vector whose components are all 0 is called the zero vector, denoted $\mathbf{0}$ (bold, italic). The zero vector has length 0. By contrast, a vector that does not have all-zero components is called a nonzero vector, and a nonzero vector has length greater than 0.

The vector for yellow light is $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$, representing the mixture of pure red and pure green light. Here you have probably already spotted vector addition, namely $\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix}$. This is a topic we will cover later in this chapter.

Yellow light corresponds to the all-ones column vector. As mentioned in the previous section, the all-ones column vector is a column vector whose components are all 1.

It's worth noting that in Figure 4(a), the all-ones column vector is exactly the diagonal of the unit square. A unit square is a square with side length 1.

⚠ in the RGB color model, each color component ranges over $[0, 1]$. The red-green plane is merely $\mathbb{R}^2$, an extremely small part of the whole. Even within the full RGB color space, the red-green plane is just one face of the RGB cube.

Vector Decomposition

On the red-green plane (the x_1x_2 plane), $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ are two very special vectors; we give them the special notation $\mathbf{e}_1$ and $\mathbf{e}_2$

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

On the “red-green” plane, the unit vector $\mathbf{e}_1$ represents the pure red vector, pointing in the positive x_1 direction; the unit vector $\mathbf{e}_2$ represents the pure green vector, pointing in the positive x_2 direction.

Clearly, $\mathbf{e}_1$ and $\mathbf{e}_2$ both have length 1; we call a vector of length 1 a unit vector.

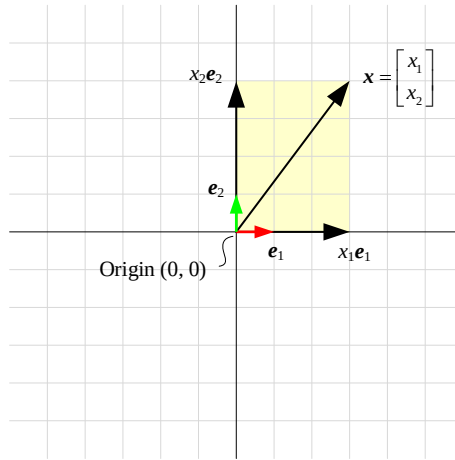


Figure 5. Writing a 2D Vector as the Sum of Two Components

As shown in Figure 5, any vector $\mathbf{x} = [x_1, x_2]^T$ in this plane can be written as

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = x_1 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + x_2 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = x_1 \mathbf{e}_1 + x_2 \mathbf{e}_2$$

This expression is essentially a vector decomposition; it also uses vector addition and scalar multiplication. These are all topics we will cover later in this chapter.

Looking at $\mathbf{e}_1$ and $\mathbf{e}_2$ weaving together into a neat square grid, it's as if the entire plane unfolds, held up by these two vectors. This is the power of span: simple, yet present everywhere, like an invisible skeleton supporting every inch of space. Keep the concept of “span” in mind — it is a cornerstone of the subject, and later on it will open the door to understanding the linear world.

▲ in the RGB color model, every color's R, G, and B components must each have a specific value; for example, the blue component of pure red is 0, and the blue component of pure green is also 0. Writing a color as a two-dimensional vector is purely for ease of understanding. Geometrically, going from three dimensions down to two amounts to a projection.

Three-Dimensional Cartesian Coordinates

Building on the Cartesian plane $\mathbb{R}^2$, we can extend further to three-dimensional Cartesian coordinates.

Specifically, if at the origin of the x_1x_2 plane we erect a new axis x_3 , perpendicular to that plane, and make the x_3 axis mutually perpendicular to the x_1 axis and the x_2 axis, while keeping the same origin, we obtain three-dimensional Cartesian coordinates, denoted $\mathbb{R}^3$.

In three-dimensional space, the position of a point is represented by an ordered triple (x, y, z) or (x_1, x_2, x_3) , and this book usually writes it as (x_1, x_2, x_3) .

A vector $\mathbf{x} = [x_1, x_2, x_3]^T$ can be viewed as a 3D vector in three-dimensional space whose tail is at $(0, 0, 0)$ and whose head is at (x_1, x_2, x_3) .

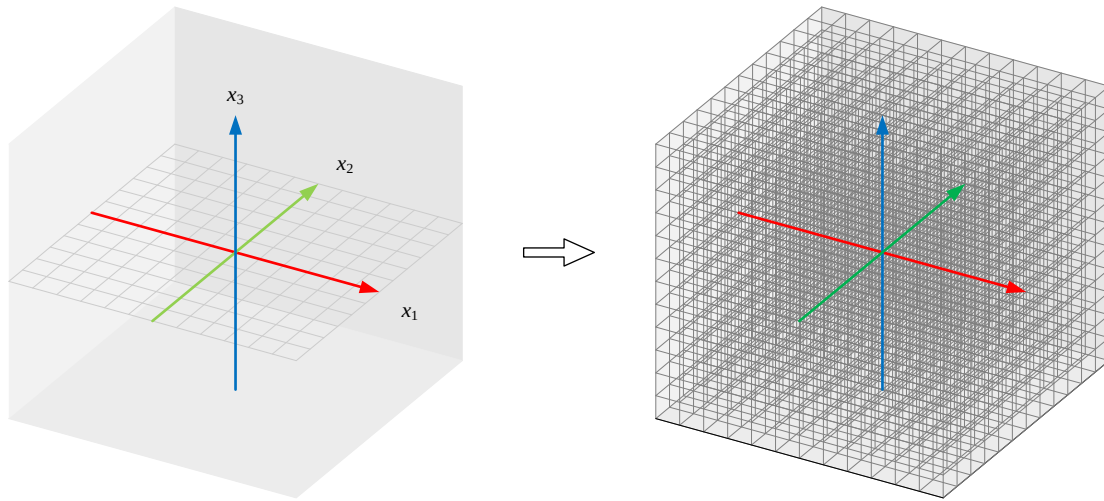


Figure 6. A Grid in Three-Dimensional Cartesian Coordinates

The three-dimensional Cartesian coordinate system used in this book follows the right-hand rule; that is, spatial directions are defined according to the right-handed convention.

Specifically, as shown in Figure 7, make a fist with your right hand, extend your four fingers so they point in the positive x_1 direction; your palm will then naturally face the positive x_2 direction; and your thumb, sticking up, points in the positive x_3 direction.

Just like the right-hand rule for the Cartesian plane, this convention ensures consistency in mathematical operations such as determinants, coordinate transformations, vector cross products, and the sense of rotation. A left-handed system would be equally valid mathematically – physics and computer graphics both use one occasionally – but a book (or a program) must pick one convention and apply it consistently throughout, since mixing the two silently flips the sign of any cross product or determinant.

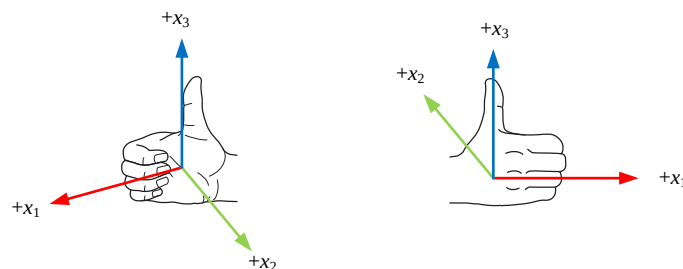
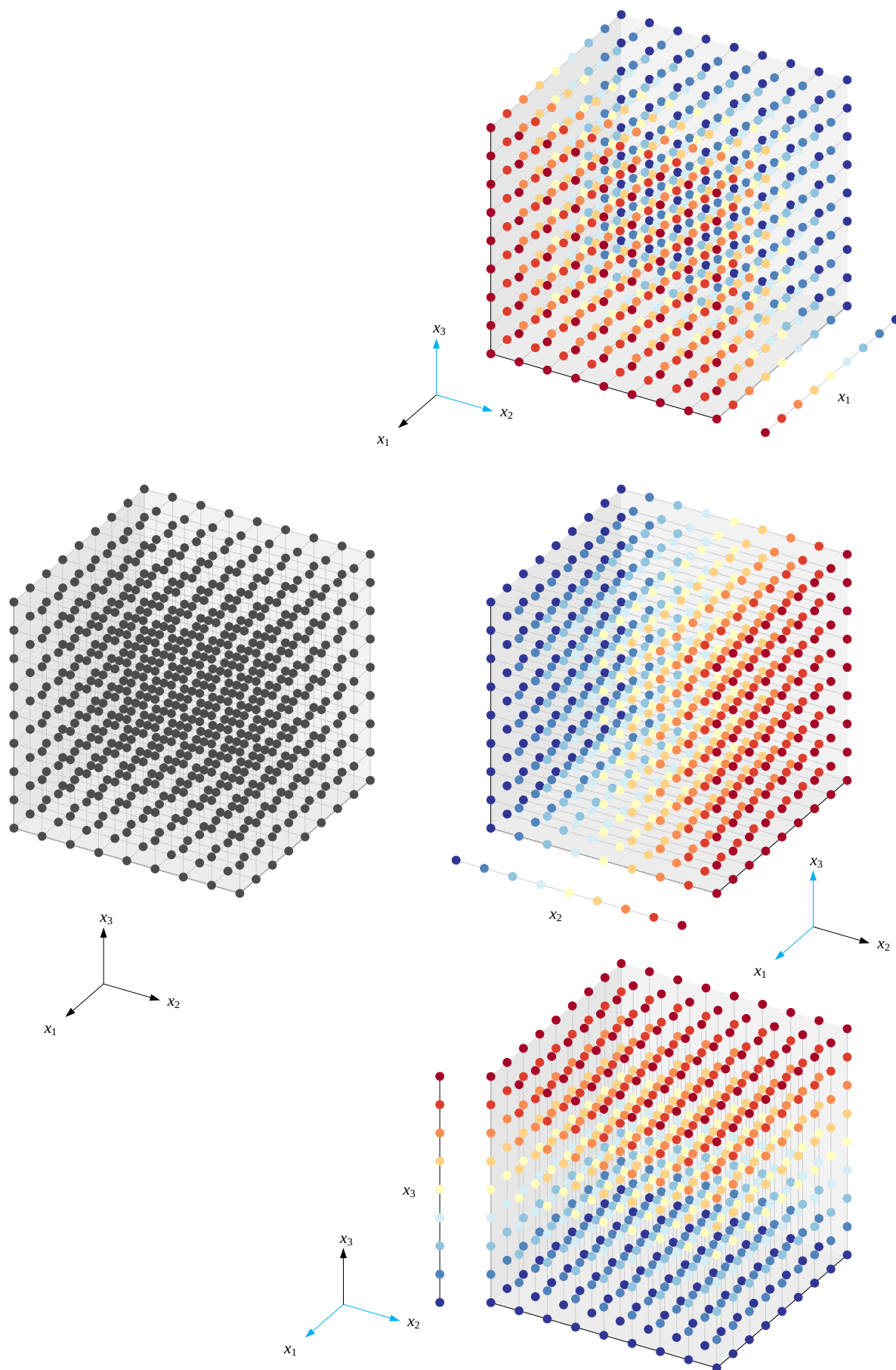


Figure 7. A Right-Handed Three-Dimensional Cartesian System

Figure 8 shows how `numpy.meshgrid()` generates three-dimensional grid data; the blue arrows indicate the directions in which the one-dimensional data are copied.



The code file LA_01_02_02.ipynb uses `numpy.meshgrid()` to generate a three-dimensional array; please study it on your own.

Figure 8. Creating Three-Dimensional Grid Data with `numpy.meshgrid()`

This PDF file is the author's draft, published to make it convenient for readers to study on mobile devices; the final content is subject to the printed edition published by Tsinghua University Press.

Copyright belongs to Tsinghua University Press. Please do not use it commercially, and please credit the source when quoting.

Code and PDF file downloads: <https://github.com/Visualize-ML>

The companion video lectures for this book are all published on Bilibili — 生姜 DrGinger: <https://space.bilibili.com/513194466>

Comments and corrections are welcome. The book's dedicated email address: jiang.visualize.ml@gmail.com

Euclidean Space

Euclidean space is a vector space equipped with the Euclidean distance — that is, straight-line distance — and it satisfies the axioms and properties of Euclidean geometry.

Simply put, a vector space is a set whose elements are called vectors, on which vector addition and scalar multiplication are defined; these two operations satisfy basic axioms such as closure, commutativity, associativity, distributivity, the existence of a zero vector, and the existence of an additive inverse for every vector. This book will cover the concept of vector spaces in detail later on.

The real number line $\mathbb{R}$, the Cartesian plane $\mathbb{R}^2$, and three-dimensional Cartesian coordinates $\mathbb{R}^3$ are all special examples of Euclidean space, corresponding respectively to one-, two-, and three-dimensional Euclidean space.

In these spaces, we can intuitively define points, lines, and planes, along with the distances and angles between them, providing a basic framework for studying problems in geometry and physics.

The concept of a vector space is extremely important. For now, it's enough to form a first impression of it — this book will return to vector spaces again and again in later chapters.

The RGB Color Model

Figure 4 shows a red-green plane woven together from the red and green components alone; it is only a small corner of the entire RGB color universe.

However, this red-green plane is merely the starting point for exploring the boundless possibilities of color; the real journey through color requires adding the dimension of blue before we can paint a far richer and more colorful world, as shown in Figure 9.

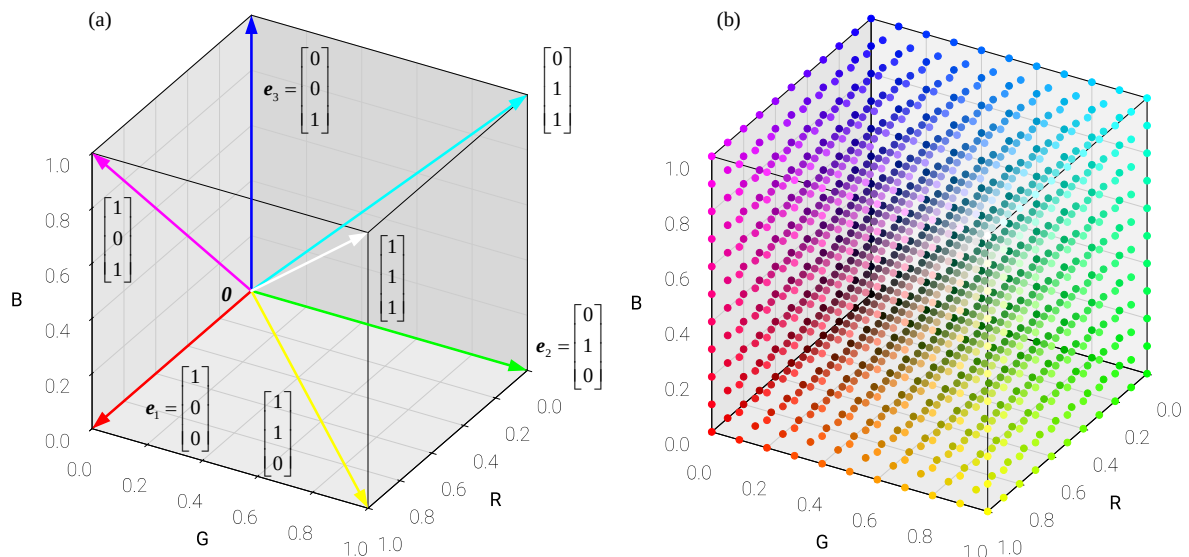


Figure 9. Arrows and Scatter Points in RGB Color Space

Just as the Cartesian plane has e_1 and e_2 , three-dimensional Cartesian coordinates also have their own special unit vectors

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \mathbf{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Here, the unit vector $\mathbf{e}_1$ points in the positive x_1 direction, the unit vector $\mathbf{e}_2$ points in the positive x_2 direction, and the unit vector $\mathbf{e}_3$ points in the positive x_3 direction.

In Figure 9(a), the RGB color space, $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ represents pure red, $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$ represents pure green, $\mathbf{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$ represents pure blue.

Mixed in sufficient amounts, red and green blend to produce vivid yellow, $\begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$; red and blue combine to form deep magenta, $\begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$; and green and blue merge to give fresh cyan, $\begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$.

In this space, the zero vector $\mathbf{0} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$ is black, and the all-ones column vector $\mathbf{1} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$ represents white.

In Figure 9(a), the all-ones column vector is exactly the diagonal of the unit cube. A unit cube is a cube with side length 1.

As shown in Figure 9(b), just as the three primary colors red, green, and blue can combine to form any color, the basis vectors $\mathbf{e}_1$, $\mathbf{e}_2$, and $\mathbf{e}_3$ of three-dimensional Cartesian coordinates can likewise combine to form any vector in three-dimensional space.

Any vector $\mathbf{x} = [x_1, x_2, x_3]^T$ in three-dimensional Cartesian coordinates can be written as

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = x_1 \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + x_2 \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + x_3 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = x_1 \mathbf{e}_1 + x_2 \mathbf{e}_2 + x_3 \mathbf{e}_3$$

This is an instance of vector decomposition, a topic we will discuss later in this chapter.

⚠ Note that, because RGB color space is bounded, strictly speaking it cannot be called a vector space — it is really more like a three-dimensional cube of colors. Even so, it can still help us build intuition for the concept of a vector space.

In the Cartesian plane, $\mathbf{e}_1$ is a 2D vector; in three-dimensional Cartesian coordinates, $\mathbf{e}_1$ is a 3D vector; and so on — in n -dimensional space, $\mathbf{e}_1$ is an n -dimensional vector.

When needed, to distinguish them, we add a superscript and denote them respectively as $\mathbf{e}_1^{(2)}$, $\mathbf{e}_1^{(3)}$, $\mathbf{e}_1^{(4)}$, and $\mathbf{e}_1^{(n)}$. For example,

$$\mathbf{e}_1^{(2)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_1^{(3)} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_1^{(4)} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_1^{(n)} = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

The Length of a Vector

With a Cartesian coordinate system in hand, we can easily compute the length of a vector.

Using the Pythagorean theorem, the magnitude of an n -dimensional vector $\mathbf{x}$ is specifically given by

$$\|\mathbf{x}\| = \|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

As an example, consider the following 2D vector

$$\mathbf{x} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

The magnitude (length) of this vector is

$$\|\mathbf{x}\| = \|\mathbf{x}\|_2 = \sqrt{3^2 + 4^2} = 5$$

In linear algebra, the magnitude (length) of a vector is often called the **L² norm**, the **Euclidean norm**, or simply the **norm**. The L2 norm is only one member of a broader family of vector norms – for example, the L1 norm sums the absolute values of the components, and the L_∞ norm takes the largest one – but unless stated otherwise, “the norm” of a vector in this book always means the L2 norm. This book will cover the concept of vector norms in detail later in this chapter.

⚠ Note that in this book, absolute value is written $|-5|$, while the vector-magnitude operator is written $\|\mathbf{x}\|$. $\|\mathbf{x}\|_2$ The subscript 2 stands for the L² norm, and it is often omitted, written simply as $\|\mathbf{x}\|$.

The length of a 3D vector $\mathbf{x}$ is

$$\|\mathbf{x}\| = \|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + x_3^2}$$

For example, take the 3D column vector $[1, 2, 2]^T$; its length (magnitude, L² norm, or norm) is

$$\|\mathbf{x}\| = \|\mathbf{x}\|_2 = \sqrt{1^2 + 2^2 + 2^2} = 3$$

Computing the Length of a Vector

Code 2 shows how to use `numpy.linalg.norm()` to compute the length of a vector (a one-dimensional array).

Code 2. Computing the Magnitude of a Vector |  LA_01_02_03.ipynb

This PDF file is the author's draft, published to make it convenient for readers to study on mobile devices; the final content is subject to the printed edition published by Tsinghua University Press.

Copyright belongs to Tsinghua University Press. Please do not use it commercially, and please credit the source when quoting.

Code and PDF file downloads: <https://github.com/Visualize-ML>

The companion video lectures for this book are all published on Bilibili — 生姜 DrGinger: <https://space.bilibili.com/513194466>

Comments and corrections are welcome. The book's dedicated email address: jiang.visualize.ml@gmail.com

```

## 初始化
import numpy as np

## 定义向量 (一维数组)
a = np.array([3, 4])
b = np.array([1, 2, 2])

## 计算向量长度 (L2范数)
length_a = np.linalg.norm(a)
np.sqrt(a[0]**2 + a[1]**2)

length_b = np.linalg.norm(b)
np.sqrt(b[0]**2 + b[1]**2 + b[2]**2)

```

Let's talk through Code 2.

- a defines two one-dimensional arrays using `numpy.array()`.
- b uses `numpy.linalg.norm()` to compute the length of a.
- c manually verifies the length of a. `a[0]` denotes a's first component, 3, and `a[1]` denotes its second component, 4. The `**` operator performs the squaring; the function `numpy.sqrt()` performs the square root — sqrt stands for square root.
- d uses `numpy.linalg.norm()` to compute the length of b.
- e manually verifies the length of b.



Please use tools such as DeepSeek/ChatGPT to complete the following exercises for this section.

Q1. What are the axioms and properties of Euclidean geometry?

Q2. What are the four quadrants of the Cartesian plane?

Q3. What are the eight octants of three-dimensional Cartesian coordinates?

Q4. Ask DeepSeek/ChatGPT to comment the following example line by line, and learn how to plot an arrow (quiver) diagram.

https://matplotlib.org/stable/gallery/images_contours_and_fields/quiver_simple_demo.html

Q5. Ask DeepSeek/ChatGPT to comment the following example line by line, and learn how to plot a scatter plot in the plane.

<https://seaborn.pydata.org/generated/seaborn.scatterplot.html>

Q6. Ask DeepSeek/ChatGPT to comment the following example line by line, and learn how to plot a 3D scatter plot.

<https://matplotlib.org/stable/gallery/mplot3d/scatter3d.html>

Q7. Use the `numpy.norm()` function to compute the length of each of the following vectors, and verify the result with the Pythagorean theorem.

► [2, 4, 4]

► [2, 3, 6]

This PDF file is the author's draft, published to make it convenient for readers to study on mobile devices; the final content is subject to the printed edition published by Tsinghua University Press.

Copyright belongs to Tsinghua University Press. Please do not use it commercially, and please credit the source when quoting.

Code and PDF file downloads: <https://github.com/Visualize-ML>

The companion video lectures for this book are all published on Bilibili — 生姜 DrGinger: <https://space.bilibili.com/513194466>

Comments and corrections are welcome. The book's dedicated email address: jiang.visualize.ml@gmail.com

► [1, 4, 8]

Q8. Please teach yourself how to use `numpy.meshgrid()` to create a three-dimensional array.

Q9. Please teach yourself how to index and slice a three-dimensional array.